SHACL

RDF 约束语言 · 规范速查

SHACL — OVERVIEW · DATA GRAPH MEETS SHAPES GRAPH

01 总览 · 数据图遇上形状图

PART 1 · OVERVIEW

SHACL 给 RDF 数据上「合同」:数据图归数据图,形状图归形状图,引擎拿组件约束逐节点比对,产出机器可读的验证报告。

两张图	数据图 + 形状图,各写各的	解耦
形状	节点形状 · 属性形状 · target 圈人	组件全表见 §04
校验	组件约束 → 引擎比对 → 报告	conforms + 违规列表
扩展与元层	SPARQL 自定义约束 · SHACL 校验自身	表达力兜底

主线:两张图解耦:数据不动,换形状即换验收标准。

本卡结构 = 本图:概览 → 组件与示例 → 报告 → 扩展 → 实操。

02 定位 · 给 RDF 数据上「合同」

PART 1 · POSITION

开放世界的 RDF 缺一样东西:约束。Shapes Constraint Language(SHACL,读作「shackle」)用一份 RDF 写的「形状图」去校验另一份「数据图」,输出结构化报告——通过、警告、不通过,条条有据。

KEY POINT

SHACL 1.0,**W3C Recommendation 2017-07-20**。语法就是 RDF(shapes graph 本身是合法 RDF 图);校验期间两份图都必须保持不可变。命名空间 `sh:` = `http://www.w3.org/ns/shacl#`。与 OWL 的分野一句话:**OWL 是推理,SHACL 是校验**——违反约束报错,而不是默默推出新类型。

SHACL 管 VALIDATION

- 数据长什么样才合格:必填、类型、取值范围、基数
- 封闭世界检查:`sh:closed` 说「除此之外不许有」
- 输出机器可读的验证报告

SHACL 不管 OUT OF SCOPE

- 推断新事实(那是 RDFS/OWL 的活,PL.02/PL.04)
- 限定词表内容管理(那是 SKOS 的活,PL.03)
- 「为什么这么建模」——它只验收,不指导

校验时机 · 一份形状图,四处上岗

入库前 图库校验档拦截	**ETL 管道** 批次抽检出报告	**发布前** CI 里跑 pySHACL	**API 网关** 写入请求即校验

03 结构模型 · 节点形状与属性形状

PART 1 · STRUCTURAL MODEL



约束焦点节点本身
无 sh:path

恰好一个 sh:path
约束沿路径够到的值节点

件套:`targetNode` · `targetClass` · `targetSubjectsOf` · `targetObjec`

\+ 隐式目标:形状同时是 `rdfs:Class` 时,自动瞄准该类的全部实例

焦点节点(focus node)由目标选出;**值节点**(value nodes)是焦点节点沿 sh:path 走到的节点。节点形状的值节点就是焦点节点自己。

两类形状**互斥**:有 sh:path 的就是属性形状,没有的就是节点形状——别混着用。

`path` 不只是 IRI

`sh:path` 接受 SPARQL 属性路径:序列 `(ex:parent ex:parent)`、反向 `[ sh:inversePath ex:parent ]`、交替 `sh:alternativePath`——校验祖孙、反向引用一行搞定。

04 约束组件全表 · SHACL Core 的弹药库

组	组件	一句话
值类型	`sh:class`	值是某类的实例(RDFS 语义下连同继承)
	`sh:datatype`	值的数据类型(xsd:string、xsd:integer…)
	`sh:nodeKind`	值是 IRI / BlankNode / Literal / BlankNodeOrIRI…
基数	`sh:minCount / sh:maxCount`	值节点个数下限/上限(仅属性形状)
	`sh:qualifiedValueShape` + Min/MaxCount	满足某形状的值至少/至多几个(如「至少一个收件人」)
数值范围	`sh:minExclusive / minInclusive`	下界(开/闭)
	`sh:maxExclusive / maxInclusive`	上界(开/闭)
字符串	`sh:minLength / maxLength`	长度上下限
	`sh:pattern` + `sh:flags`	正则(XPath 正则,flags 可加 i 大小写不敏感)
	`sh:languageIn`	语言标签白名单(["zh", "en"])
	`sh:uniqueLang`	同一语言标签至多一个值(true)
属性对	`sh:equals`	与另一属性的值完全相同
	`sh:disjoint`	与另一属性无公共值
	`sh:lessThan / lessThanOrEquals`	本属性值严格小于(≤)另一属性值——startDate < endDate
逻辑组合	`sh:not [shape]`	值节点不满足该形状
	`sh:and / sh:or`	形状列表全满足 / 至少一个
	`sh:xone`	恰好满足其一(异或)——「二选一必填」神器
	`sh:hasValue`	值里必须含某个特定值
形状引用	`sh:node [shape]`	值节点整体满足另一节点形状(嵌套复用)
	`sh:property`	挂子属性形状
封闭	`sh:closed true`	除声明过的属性外不许有其他谓语
	`sh:ignoredProperties`	closed 的豁免名单(rdf:type 常在此)
枚举	`sh:in ( "低" "中" "高" )`	值必须在列表内

经典组合 · 三条最高频的形状

```
# 1. 必填且唯一且是字符串
[ sh:path foaf:name ; sh:datatype xsd:string ;
  sh:minCount 1 ; sh:maxCount 1 ]

# 2. 多语言名:每个标签至多一个,语言限定中英
[ sh:path rdfs:label ; sh:uniqueLang true ;
  sh:languageIn ( "zh" "en" ) ]

# 3. 受控字段:枚举 + 模式
[ sh:path :status ; sh:in ( "draft" "published" "retracted" ) ]
[ sh:path :orcid ; sh:pattern "[0-9]{4}-[0-9]{4}-[0-9]{4}-[0-9]{3}[0-9X]" ]
```

05 一份完整形状图 · Turtle

PART 2 · A COMPLETE SHAPES GRAPH

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix :     <http://example.org/shapes#> .

:PersonShape a sh:NodeShape ;
    sh:targetClass :Person ;            # 瞄准所有 Person 实例
    sh:closed true ;                    # 未声明属性一律违规
    sh:ignoredProperties ( rdf:type ) ;
    sh:property [                       # 属性形状:内联匿名节点
        sh:path foaf:name ;
        sh:datatype xsd:string ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:minLength 1 ] ;
    sh:property [
        sh:path :age ;
        sh:datatype xsd:integer ;
        sh:minInclusive 0 ; sh:maxInclusive 150 ;
        sh:maxCount 1 ] ;
    sh:property [
        sh:path :knows ;
        sh:class :Person ] ;            # 只能认识 Person
    sh:xone (                            # 二选一:邮箱或电话至少一项
        [ sh:property [ sh:path :email ; sh:minCount 1 ] ]
        [ sh:property [ sh:path :phone ; sh:minCount 1 ] ] ) .

# 数据图里 :alice 少了 name 且 age=200 → 两条 violation
```

06 验证报告 · 机器可读的验收单

PART 2 · VALIDATION REPORT

```
# 校验输出:一份报告图
[] a sh:ValidationReport ;
    sh:conforms false ;                # 总开关:有没有任何违规
    sh:result [
        a sh:ValidationResult ;
        sh:focusNode :alice ;           # 谁违规
        sh:resultPath foaf:name ;       # 哪条路径
        sh:value "zh"^^xsd:integer ;    # 什么值
        sh:resultSeverity sh:Violation ; # 多严重
        sh:sourceConstraintComponent sh:DatatypeConstraintComponent ;
        sh:sourceShape :PersonShape-name ;
        sh:resultMessage "必须是字符串" ] .
```

Violation 默认严重级 | Warning 提醒不拦截 | Info 纯提示

ValidationResult 字段 · 必选与可选

必选	可选
sh:focusNode · sh:resultSeverity · sh:sourceConstraintComponent	sh:value · sh:resultPath · sh:sourceShape · sh:resultMessage · sh:detail(嵌套结果)

严重级可调

约束上挂 sh:severity sh:Warning 就把某条规则降级为警告——同一份形状图,入库校验用全 Violation,数据清洗用 Warning+Info 全开。

07 SHACL-SPARQL · 表达力不够就写查询

Core 组件覆盖不到的约束,用 SPARQL 写。Core 实现必须忽略 `sh:sparql`——用它就要认准带 SPARQL 引擎的校验器。

```
# 自定义约束:国家必须出现在维基数据里(示意)
ex:CountryShape a sh:NodeShape ;
    sh:targetObjectsOf :inCountry ;
    sh:sparql [
        sh:message "国家需要 instance of (Q5) 声明" ;
        sh:select """
            SELECT $this
            WHERE { FILTER NOT EXISTS { $this wdt:P31 wd:Q3624078 } }
        """ ] .

# 预绑定变量:$this = 焦点节点;路径约束另有 $path / $value
```

扩展机制	说明
SPARQL 约束	`sh:sparql` + SELECT(报违规)或 ASK(报通过)
自定义组件	`sh:parameter` 声明参数 + `sh:labelTemplate` + 验证器——造自己的「组件」给别人用
SPARQL 目标	`sh:targetSPARQL`:用 SELECT 自己选焦点节点

qualified 的经典用法 · 披萨规则

```
# 一份披萨:至少一个非素菜配料(qualified 只数「满足形状」的值)
:PizzaShape sh:property [
    sh:path :hasTopping ;
    sh:qualifiedValueShape [
        sh:not [ sh:class :VegetarianTopping ] ] ;
    sh:qualifiedMinCount 1 ] .
```

08 SHACL-SHACL · 用 SHACL 校验 SHACL

规范附录自带一份「元形状」:校验你的形状图本身写得对不对——组件误用、类型乱挂都逃不掉。

> **实用结论**
> 上线前把形状图过一遍 SHACL-SHACL:`sh:shapesGraphWellFormed` 为 true 才算语法健康。形状写错不是数据错,错误会更隐蔽——这一步别省。

形状图 `shapes graph`
- RDF 图,内容是形状与约束
- 校验期间不可变
- 可复用、可导入、可组合

数据图 `data graph`
- 被校验的对象
- 校验期间不可变
- 与形状图可以是同一张图(不推荐混)

09 实操坑清单

PART 4 · PITFALLS

01 sh:class 看的是 **rdf:type 的传递语义**(RDFS 下连同子类),sh:datatype 是严格词法比对——「看着像」不等于「校得过」。

02 字面量 "5" 与 5 不是一回事:数据类型没对上,sh:minInclusive 直接违规——先统一数据类型再上形状。

03 节点形状与属性形状**互斥**:加了 sh:path 就变属性形状,minCount 只在属性形状里有效。

04 sh:closed 是按**谓语**封闭:路径里没声明的谓语一律违规——rdf:type 记得进 ignoredProperties。

05 递归形状(sh:node 自我引用形成环)的行为规范未定义——设计形状层级时避免循环引用。

06 sh:xone 是「恰好一个」,sh:or 是「至少一个」——「二选一必填」用 xone,用 or 会放过全选。

07 合格性是逐值判定的:一条约束违规生成一条 result,别指望报告里自动聚合。

08 校验期间图必须不可变:边校验边改数据的管线是未定义行为,先出报告再动数据。

09 OWL 公理不参与 SHACL 校验——想让 sh:class 认子类,校验器得开 RDFS 推理档,或显式补类型。

10 形状图与数据图分开存放:混在一张图里,校验目标会把形状也「验证」一遍。

10 工具速览 · 校验从哪进

PART 4 · TOOLING

工具	路线
pySHACL (Python)	rdflib 生态标配,Core + 尽量 SPARQL 扩展,CLI 友好
TopBraid / Composer	SHACL 诞生地,编辑校验一体化
Jena SHACL / RDFLib	Java 生态管线内嵌校验
SHACL Playground	浏览器里在线试形状,教学演示利器
GraphDB / Stardog	图库内置校验档位,入库前拦截

与 OWL 的组合拳

常见管线:OWL 推理(补类型、消歧)→ SHACL 校验(拦垃圾)。两者按此顺序各司其职——顺序反了,推理会「修好」本该报错的违规。

三个随抄随用的模板

```
# 模板 A:封闭实体(配合 ignoredProperties 放行系统谓语)
:BookShape sh:closed true ; sh:ignoredProperties ( rdf:type rdfs:label ) .

# 模板 B:条件必填(不是书,就必须有 DOI)—xone 双分支惯用法
:PubShape sh:xone (
    [ sh:property [ sh:path rdf:type ; sh:hasValue :Book ] ]
    [ sh:property [ sh:path :doi ; sh:minCount 1 ] ] )

# 模板 C:引用完整性(关系值整体过另一形状)
sh:property [ sh:path :author ; sh:node :PersonShape ] .
```